

Exercise 1.1: Basic Java Programs

Lab Objectives

- Familiarise yourself with Eclipse IDE for Java
- Start to write basic Java programs
- Understand the Primitive Data Types
- Work with Control Structures and Arrays in Java

Basic Java Programs

Eclipse is the development environment in which you will be developing, compiling and running Java programs. The induction materials should already have familiarised you with the IDE.

Question 1 To get started with this lab exercise, create an "EclipseWorkspaces" folder on your H drive (or a location of your choice on your own laptop). Within this, create a new folder called "CTEC2710_OO" (which we will use as the workspace for this module), then open up Eclipse, selecting this as your workspace. Download the existing Java Project required for this lab and extract it from the "**BasicJavaPrograms.zip**" file. The project still needs to be imported into your Eclipse workspace. To do this, you need to go to **File > Import... > General > Existing Projects into Workspace > Browse** and locate the BasicJavaPrograms project folder (ensure you select the root folder i.e. the one directly above the bin and src folders). Next select "Copy projects into workspace" and press **Finish**.

- a) Within the **primitives** package of your project create a new Java class file called "**HelloWorld**", and copy the following code into it:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
  
}
```

- b) Build (compile) and Run the program within the Eclipse environment, you should see the message "Hello World" output in the console.

Notes - Basic concepts:

- **class** - Every Java program is a class, and the file name must match the class name.
- **main(...)** - A Java class is executable if it has a **main()** method, all executable code should be placed within this.
- **System.out.println(...);** - outputs the String parameter provided to the console.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/getStarted/application/index.html>

Note: you will find further reading resources at the end of this week's slides, e.g. associated Oracle tutorials, and will also find useful syntax examples within the slides themselves too.

Primitive data types

Variables allow you to store data within your programs and the data type of a variable determines what it can store. Java is an Object-Oriented programming language, which means that almost every variable used within Java programs will store a reference to an Object and its type will be that of a Class. The exception to this are primitive type variables.

Primitive types form a special group of commonly used data types that exist within Java, that primarily are used for storing numeric values and carrying out numeric computations.

There are eight primitive data types in Java.

byte, short, int, long, float, double, char, boolean.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html><https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html>

Question 2 Continue working in the same project's primitives package and open "Converter.java". Study the code and then run the file to see what it does.

- Change the data type of *f* and *c* to use **double** instead of **int**.
- Compare the answers - notice how the integer division truncates.

Notes - Primitive types:

- The most commonly used primitive data types are **int** for whole numbers, **double** for decimal point numbers and **boolean**, which stores *true* or *false*.
- Primitive data types can be concatenated into a String using the **+** operator. This adds their value to the String, as can be seen in the Converter program.
- The arithmetic operators in Java are similar to the standard ones within Scala: **+** **-** ***** **/** **%**

Please turn over...

Control structures

The execution of code in a program is dictated by a combination of sequence, selection and iteration. Sequence is when code executes line by line in order, selection is when a decision is taken that determines a pathway through a program and iteration is when a block of code is repeated.

You should already be familiar with **if-else** and **switch** statements used to make decisions within programs in addition to **for** and **while** loops used for iteration.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>

Question 3 Within the **primitives** package open the "**ConverterNew.java**" program.

- Study the code and run the file to see what it does.
- Change the value assigned to the boolean variable *b* to be *false* and run the program again.
- Look at what happens if you change the value from 9.0 to 9 in the conversion formula within the else block. Why do you think this is?

Question 4 Now within the **controlstructures** package create a new class called "**GradeMark**". In the main method do the following:

- Assign a value between 0 and 100 to a variable named *mark* with type `int`.
- Use an **if-else** statement to output "Pass" if mark is 40 or more, otherwise output "Fail".
- Extend the if-else statement in the program above to output Fail, Pass, Merit, or Distinction depending on whether *mark* is less than 40, 40-59, 60-69, or 70+. There may be helpful syntax examples in the slides, or Oracle tutorials. **Hint:** You may wish to look at the IfElseDemo program [here](#) to help you get started.

Notes - Conditional operators:

- In Java the **six** Relational operator: `==` `!=` `>` `>=` `<` `<=`
- Java also has **three** Boolean operators: `&&` `||` `!`
- These nine operators can be combined into complex expressions but will always result in a value of `true` or `false`.

Please turn over...

Question 5 Within the same package create a new class called "**DaysOfWeek**" and then in the main method:

- (a) Assign a value between 1 - 7 to a variable named *day* of type int.
- (b) Use a **switch** statement to output "Monday" if the day is 1, "Tuesday" if 2, etc, or "Unknown day" for any number outside this range. You can look at the slides to assist you with syntax.
- (c) Now update the program to have a second switch that outputs "Weekday" if the day is between 1-5 and "Weekend" if it is 6-7, otherwise "Unknown day". You may like to look at [this](#) tutorial to see how multiple cases can produce the same outcome.
- (d) A better version of this program that uses [Enum Types](#) can be seen in "**DaysOfWeekEnums.java**" that you have been given inside the controlstructures package.

Question 6 Within the controlstructures package open "**TimesTable.java**".

- (a) Study the code and run the file to see what it does.
- (b) Change the program so that it uses a **while** loop instead of a for loop. To do this you will need to declare and initialise *i* outside of the loop.
- (c) Using another counter, e.g. *j*, extend the program above to write out the times tables for all the numbers from 1 to 12 in a table. You will need an inner loop to do this and you will not need the variable named *number*.

Arrays in Java

In simple terms an array is used to store a sequence of values of the same type. This means that an array can store a collection of primitive types, e.g. int, or reference types, e.g. String (which you will see more of next week). The length of an array is fixed once it has been created and can be accessed via a public field called 'length'.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>

Question 7 In the **arrays** package open "**NumberArrayDemo.java**".

Study the code and comments. You will see how an array has been declared with each element being initialised. Several tasks have then been carried out and the vast majority of the syntax should be familiar. You will notice a Scanner being used to read input, and will work with this further next week.